

# 차시별 실습 코드 관리

수업 원본 저장소는 `src/main.cpp` **한 파일**에 실습 코드를 씁니다. 그래서 새 차시를 할 때마다 이전 코드를 지우고 덮어써야 합니다. 이 문서대로 바꾸면 **차시마다 폴더를 따로** 두고, 원하는 차시를 골라 언제든지 다시 업로드할 수 있습니다.

이 문서의 "변경 후" 구조는 **수업 공식 구성이 아니라**, 원본 저장소를 차시별로 쓰기 위해 개인적으로 정리한 방식입니다. 수업 안내와 다르면 **수업 안내를 따르세요**.

## 1 무엇이 바뀌나

### 원본

```
mc2026/
├── platformio.ini
└── src/
    └── main.cpp
```

← 모든 차시를 이 파일 하나에 덮어씀

### 변경 후

```
mc2026/
├── platformio.ini
└── src/
    ├── w01_xxx/
    │   └── main.cpp
    ├── w02_xxx/
    │   └── main.cpp
    └── w03_xxx/
        └── main.cpp
```

← 차시 목록을 여기에 등록  
← 1차시  
← 2차시  
← 새 차시는 폴더째 추가

xxx 자리에는 그 차시의 실습 주제를 영문으로 씁니다. (→ 5절 이름 규칙)  
이 문서의 나머지 부분에서도 xxx 로 표기합니다.  
이 문서에서 **주황색 글자**는 본인이 정한 이름으로 바꿔 쓸 수 있는 부분입니다.

- 폴더 하나 = 차시 하나 = `platformio.ini` 의 환경( `[env:w01_xxx]` ) 하나
- 업로드할 때 **고른 차시 폴더만** 컴파일됩니다.

### 왜 `platformio.ini` 도 고쳐야 하나요?

`src/` 안의 `.cpp` 파일은 기본적으로 **전부 한꺼번에** 컴파일됩니다. `main.cpp` 가 여러 개면 `setup()` / `loop()` 이 겹쳐서 빌드가 실패합니다. 그래서 차시마다 "이 폴더만 컴파일하라"고 `platformio.ini` 에 알려 줘야 합니다.

## 2 처음 한 번 설정하기

아래 순서대로 진행합니다. **[필수]** 는 반드시 해야 하고, **[권장]** 은 안 해도 동작하지만 하면 편합니다.

**이름에 대해:** 아래 단계의 `w01_xxx` 에서 `xxx` 는 본인이 정하는 이름입니다. 어떤 이름이든 다음 네 곳을 모두 같은 이름으로 맞춰야 합니다.

1. 폴더 이름 — `src/w01_xxx/`
2. 환경 이름 — `[env:w01_xxx]`
3. 필터 — `build_src_filter = +<w01_xxx/>`
4. 기본 차시 — `default_envs = w01_xxx`

하나라도 다르면 `Unknown environment names` 나 `undefined reference to 'setup'` 오류가 납니다. 단, `[env:uno]` 의 `uno` 는 점검 스크립트가 쓰는 고정 이름이므로 바꾸지 마세요.

### 단계 1 **[필수]** — `main.cpp` 를 1차시 폴더로 옮기기

1. VS Code 왼쪽 탐색기에서 `src` 폴더를 오른쪽 클릭 → 새 폴더 → `w01_xxx` 입력
2. `src/main.cpp` 를 `src/w01_xxx/` 안으로 끌어다 놓습니다.

```
src/main.cpp → src/w01_xxx/main.cpp
```

`src/` 바로 아래에 `main.cpp` 가 남아 있으면 모든 차시와 겹쳐서 `multiple definition of 'setup'` 오류가 납니다.

### 단계 2 **[권장]** — `board = uno` 를 공통 설정으로 옮기기

`platformio.ini` 의 `[env]` 부분에서 `framework = arduino` 바로 아래에 한 줄을 추가합니다.

원본

```
framework = arduino
```

변경 후

```
framework = arduino
board = uno
```

모든 차시가 이 설정을 물려받으므로, 차시마다 `board = uno` 를 쓸 필요가 없어집니다.

### 단계 3 **[필수]** — `[env:uno]` 에 필터 추가

파일 아래쪽의 `[env:uno]` 를 찾아 고칩니다. **지우면 안 됩니다.** 설치 점검 스크립트(`doctor`)가 이 환경으로 빌드 테스트를 합니다.

원본

```
[env:uno]
board = uno
```

변경 후 (단계 2를 했다면)

```
[env:uno]
build_src_filter = +<w01_xxx/>
```

변경 후 (단계 2를 안 했다면)

```
[env:uno]
board = uno
build_src_filter = +<w01_xxx/>
```

`[env:uno]` 는 점검용으로 **1차시 하나만** 빌드합니다. 새 차시를 추가해도 `[env:uno]` 는 그대로 둡니다. 여기에 다른 차시를 덧붙이면 `setup()` 이 겹쳐서 `multiple definition` 오류가 납니다.

**단계 4** 필수 — 1차시 환경 추가

platformio.ini 맨 아래에 추가합니다.

```
[env:w01_XXX]
build_src_filter = +<w01_XXX/>
```

단계 2를 안 했다면 이 블록에도 board = uno 줄을 넣어야 합니다.

**단계 5** 권장 — 기본 차시 지정

파일 맨 위 [platformio] 부분을 고칩니다.

원본

```
[platformio]
default_envs = uno
```

변경 후

```
[platformio]
default_envs = w01_XXX
```

환경을 따로 고르지 않았을 때( Default ) 이 차시가 업로드됩니다. 새 차시를 시작할 때마다 이 값을 그 차시로 바꿔 두면 편합니다.

**단계 6** 권장 — 정적 분석 범위

[env] 부분의 check\_src\_filters 를 고칩니다.

원본

```
check_src_filters = +<src/*>
```

변경 후

```
check_src_filters = +<src/>
```

pio check (정적 분석)가 차시 폴더 안까지 확실히 검사하도록 범위를 넓힙니다. 빌드·업로드에는 영향이 없으므로, 수업에서 pio check 를 쓰지 않으면 건너뛰어도 됩니다.

**유지** 건드리지 않는 것

- 버전 고정: platform , platform\_packages
- 시리얼 모니터: monitor\_speed , monitor\_filters , monitor\_echo
- 경고 옵션: build\_src\_flags
- 포트 설정 주석, lib\_deps 주석

**단계 7 — 확인**

1. 하단 상태 표시줄에서 ✓(빌드)를 눌러 SUCCESS 가 나오는지 확인합니다.
2. 설치 점검 스크립트를 한 번 실행해 점검 통과 가 나오는지 확인합니다.

- Windows: powershell -ExecutionPolicy Bypass -File .\scripts\doctor.ps1
- macOS: bash scripts/doctor.sh

원본 저장소의 main.cpp 는 빈 파일로 배포됩니다. 코드를 아직 쓰지 않았다면 undefined reference to 'setup' 오류가 나는데, 설정이 잘못된 게 아니라 코드가 없어서입니다. setup() / loop() 을 먼저 작성한 뒤 빌드하세요.

## 한눈에 보기 — 바뀐 부분만

```
[platformio]
default_envs = w01_xxx ; 단계 5 [권장]

[env]
; (버전 고정 · 시리얼 모니터 · 경고 옵션 등 기존 설정은 그대로)
framework = arduino
board = uno ; 단계 2 [권장]
check_src_filters = +<src/> ; 단계 6 [권장]

[env:uno]
build_src_filter = +<w01_xxx/> ; 단계 3 [필수]

[env:w01_xxx]
build_src_filter = +<w01_xxx/> ; 단계 4 [필수]
```

### 3 새 차시 추가하기

예: 2차시를 w02\_xxx 라는 이름으로 추가 (2절의 네 곳 규칙은 동일)

#### ① 폴더와 파일 만들기

src/w02\_xxx/main.cpp 를 만들고 코드를 작성합니다.

```
#include <Arduino.h>

void setup() {
}

void loop() {
}
```

#### ② platformio.ini 맨 아래에 등록하기

폴더 이름과 환경 이름을 똑같이 맞추세요.

```
[env:w02_xxx]
build_src_filter = +<w02_xxx/>
```

#### ③ (선택) 기본 차시로 지정

```
default_envs = w02_xxx
```

#### ④ 환경 선택 후 업로드

상태 표시줄에서 env:w02\_xxx 을 고르고 업로드합니다. (→ 4절) 목록에 새 차시가 안 보이면

Ctrl + Shift + P (macOS: Cmd + Shift + P) → Developer: Reload Window 를 실행하세요.

## 4 업로드할 차시 고르기

PlatformIO 는 **지금 열려 있는 파일이 아니라, 선택된 환경을** 업로드합니다. 2차시 파일을 열어 놓고 업로드해도 환경이 1차시로 선택돼 있으면 1차시가 올라갑니다.

### 방법 A 하단 상태 표시줄 (기본)

1. VS Code 맨 아래 파란 줄에서 `Default (mc2026)` 라고 쓰인 부분을 클릭합니다.
2. 목록에서 `env:w02_XXX` 처럼 원하는 차시를 고릅니다.
3. 평소처럼  $\checkmark$ (빌드) /  $\rightarrow$ (업로드) /  $\uparrow$ (시리얼 모니터) 를 누릅니다.

한 번 고르면 **다른 차시로 바꿀 때까지 유지**됩니다.

### 방법 B 왼쪽 PlatformIO 사이드바

1. 왼쪽 세로 메뉴의 **개미 아이콘 (PlatformIO)** 을 클릭합니다.
2. **PROJECT TASKS** 아래에 차시별 항목이 있습니다.
3. `w02_XXX`  $\rightarrow$  `General`  $\rightarrow$  **Upload** 를 클릭합니다.

```
PROJECT TASKS
├─ w01_XXX
│   └─ General - Build / Upload / Monitor
├─ w02_XXX
│   └─ General - Build / Upload / Monitor
```

차시 이름을 보고 직접 누르므로 헛갈릴 일이 가장 적습니다.

### Default 는 무엇인가

상태 표시줄이 `Default` 일 때는 `platformio.ini` 맨 위의 `default_envs` 에 적힌 차시가 업로드됩니다.

`default_envs` 줄을 지우지는 마세요. 지우면 `Default` 가 "모든 차시" 가 되어, 업로드 시 모든 차시를 차례로 올리고 결국 마지막 차시만 보드에 남습니다.

## 5 이름 규칙

| 규칙                                                | 좋은 예                                                                   | 나쁜 예                                                               |
|---------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>w</code> + 두 자리 숫자 + <code>_</code> + 영문 주제 | <code>w03_button</code>                                                | 3차시 , week3 button                                                 |
| 영문 소문자·숫자· <code>_</code> 만 사용                    | <code>w04_pwm_fade</code>                                              | w04-PWM Fade                                                       |
| 폴더 이름 = 환경 이름                                     | <code>src/w05_adc/</code> $\leftrightarrow$ <code>[env:w05_adc]</code> | <code>src/w05_adc/</code> $\leftrightarrow$ <code>[env:adc]</code> |
| 파일 이름은 항상 <code>main.cpp</code>                   | <code>src/w05_adc/main.cpp</code>                                      | <code>src/w05_adc/adc.cpp</code> 만 있음                              |

한글·공백이 들어가면 Windows 에서 빌드가 깨지는 경우가 있습니다.